

Spring Boot – Injection de dépendances et accès aux données

Spring Boot fournit des mécanismes simples pour connecter les composants entre eux et accéder à une base de données.

Injection de dépendances (Spring Core)

L'injection de dépendances consiste à **ne pas créer soi-même ses objets**, mais à laisser Spring le faire.

Idée clé :

- Une classe dépend d'une autre
- Spring fournit automatiquement cette dépendance

Avantages :

- Code plus simple
- Moins couplé
- Plus facile à tester

Exemple sans injection (à éviter)

```
public class ServiceJeu {  
    private ScoreService scoreService = new ScoreService();  
}
```

Problème :

- Dépendance forte
- Difficulté de maintenance

Exemple avec injection Spring

```
import org.springframework.stereotype.Service;

@Service
public class ServiceJeu {

    private final ScoreService scoreService;

    public ServiceJeu(ScoreService scoreService) {
        this.scoreService = scoreService;
    }
}
```

Spring :

- Crée les objets
- Gère leur cycle de vie
- Injecte automatiquement les dépendances

Annotations principales

- `@Component` : composant Spring générique
- `@Service` : logique métier
- `@Repository` : accès aux données
- `@Autowired` : injection automatique (souvent implicite)

PS: En soit **@Component** pourrait techniquement être utilisé pour tout car les autres annotations hérite de **@Component**

Introduction aux Repository

Un **Repository** est une interface qui permet d'accéder à la base de données.

Objectif :

- Éviter d'écrire du SQL manuellement
- Manipuler des objets Java

Spring Data fournit des repositories prêts à l'emploi.

Exemple d'entité

```
import jakarta.persistence.Entity;  
import jakarta.persistence.Id;  
  
@Entity  
public class Joueur {  
  
    @Id  
    public Long id;  
    public String nom;  
    public int score;  
}
```

Exemple de Repository

```
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
  
@Repository  
public interface JoueurRepository extends JpaRepository<Joueur, Long> {  
}
```

Spring génère automatiquement :

- save
- findAll
- findById
- deleteById

Connexion à une base MySQL

Dépendances nécessaires :

- Spring Web
- Spring Data JPA
- MySQL Driver

Configuration dans `application.properties` :

```
spring.datasource.url=jdbc:mysql://localhost:3306/ma_bdd  
spring.datasource.username=root  
spring.datasource.password=motdepasse  
  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

Spring :

- Se connecte à la base
- Crée ou met à jour les tables
- Mappe les objets Java aux tables

Utiliser le Repository dans un Controller

Le repository est injecté dans le contrôleur.

```
@RestController
public class JoueurController {

    private final JoueurRepository joueurRepository;

    public JoueurController(JoueurRepository joueurRepository) {
        this.joueurRepository = joueurRepository;
    }

    @GetMapping("/joueurs")
    public List<Joueur> getJoueurs() {
        return joueurRepository.findAll();
    }
}
```

Conclusion

- Spring gère la création et l'injection des objets
- Les repositories simplifient l'accès aux données
- MySQL est connecté via une configuration simple
- Les controllers consomment les repositories sans dépendance directe à la base